



SCHOOL HOMEWORK - 2024

CLASS: INDUSTRIAL CONTROL SYSTEMS
SECURITY

SUBJECT: TOKENS — SNORT — MITRE ATT&CK — ZEEK & RITA —
SILK — THREAT HUNTING INVESTIGATION

FRÉDÉRIC PERRON



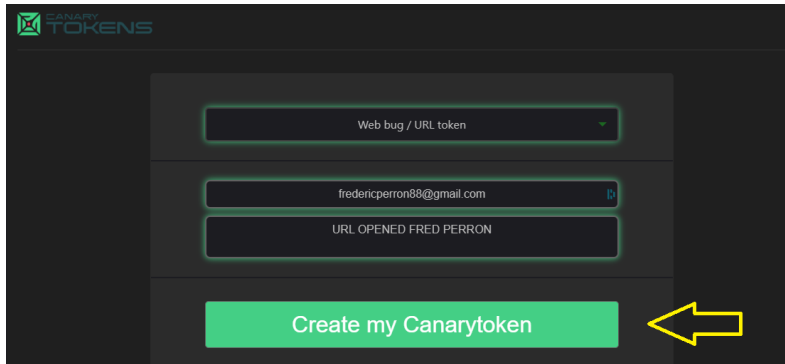
Table of Contents

Exercise No 1 – Detection Tokens.....	1
Exercise No 2 - Rule Snort.....	5
Exercise No 3 – Mitre Attack Navigator	11
Exercise No 4 – Zeek & RITA	14
Exercise No 5 – SiLK	17
Exercise No 6 – Threat Hunting investigation. max 6 pages.....	20

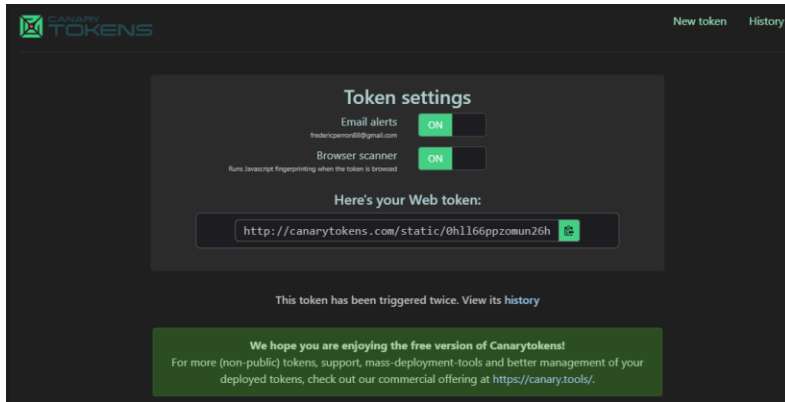
Exercise No 1 – Detection Tokens (20p)

Using the website <https://canarytokens.org/generate> create scenarios and use two different tokens. If possible, include your name in the chosen solution. Points are awarded based on the complexity of the solution.

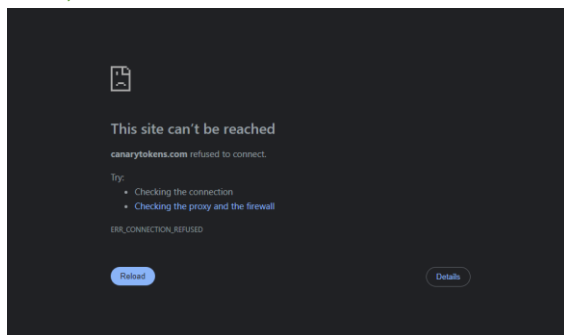
The first scenario is very simple but can be extremely useful. I start by creating a token using the Web Bug URL option.



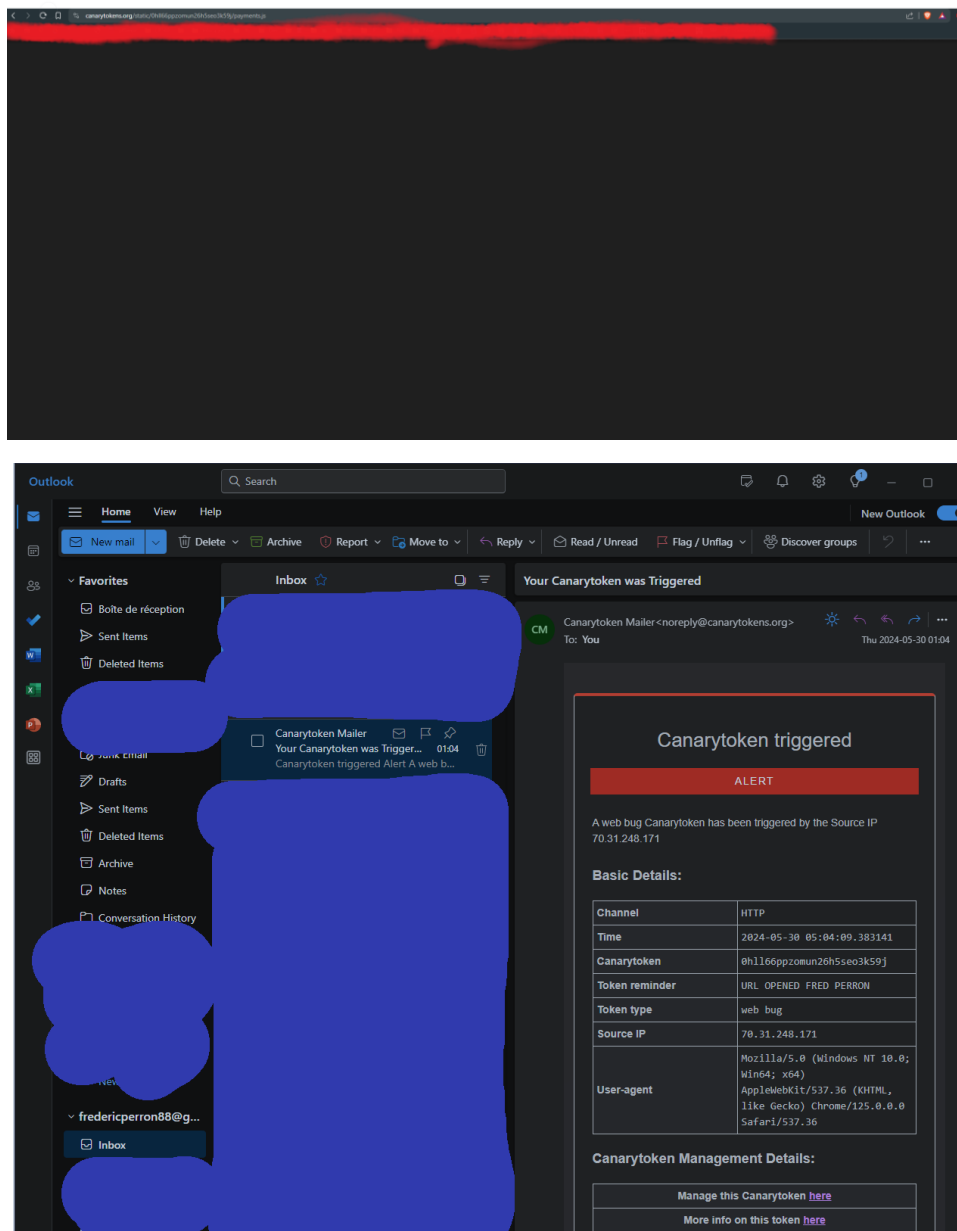
We can then view the generated link, along with options to modify its settings



Next, we take this link and visit it in another browser window. **Note:** The URL must use **.org** instead of **.com**, otherwise it results in a connection error. See image below



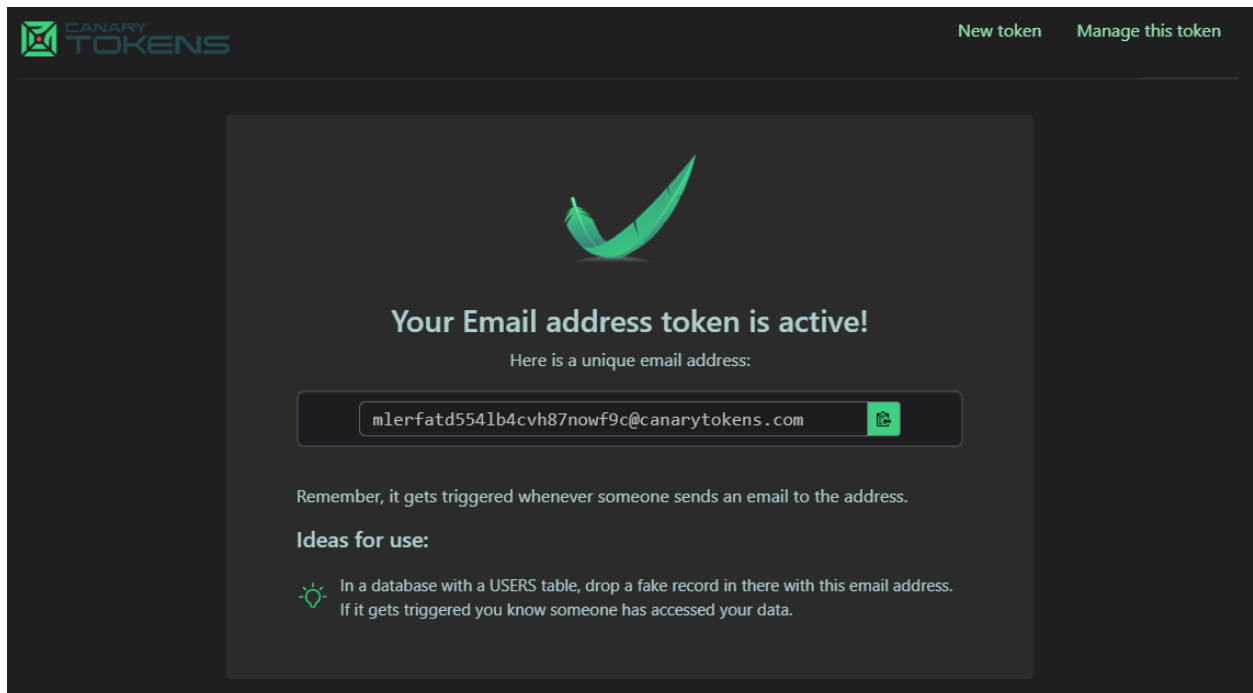
When we modify the same URL to use **.org**, we finally obtain the expected blank page, and the alerts are successfully triggered.



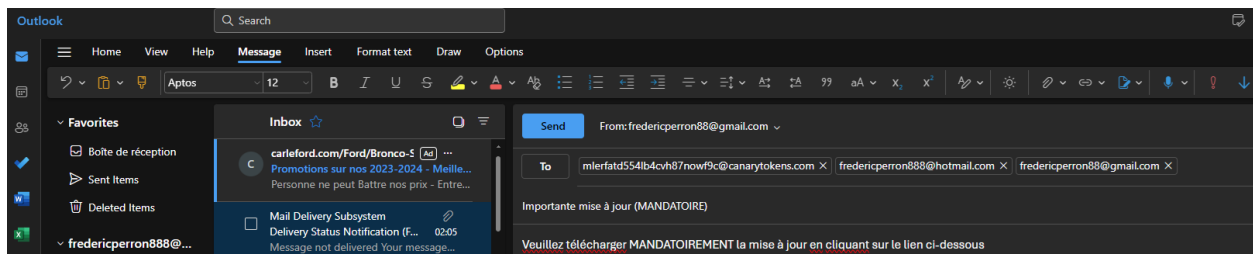
The **Web Bug URL** token is a powerful monitoring tool that can be used in a variety of security scenarios. In this case, where the goal is to detect unauthorized access attempts to a sensitive document, the Web Bug URL provides an effective solution. By discreetly embedding this link into documents, emails, or other communication channels, we can create decoys that immediately trigger alerts when accessed, indicating potentially suspicious activity.

Second scenario is also simple. We use a temporary email address generated by Canarytokens to trigger alerts whenever emails are sent to that address.

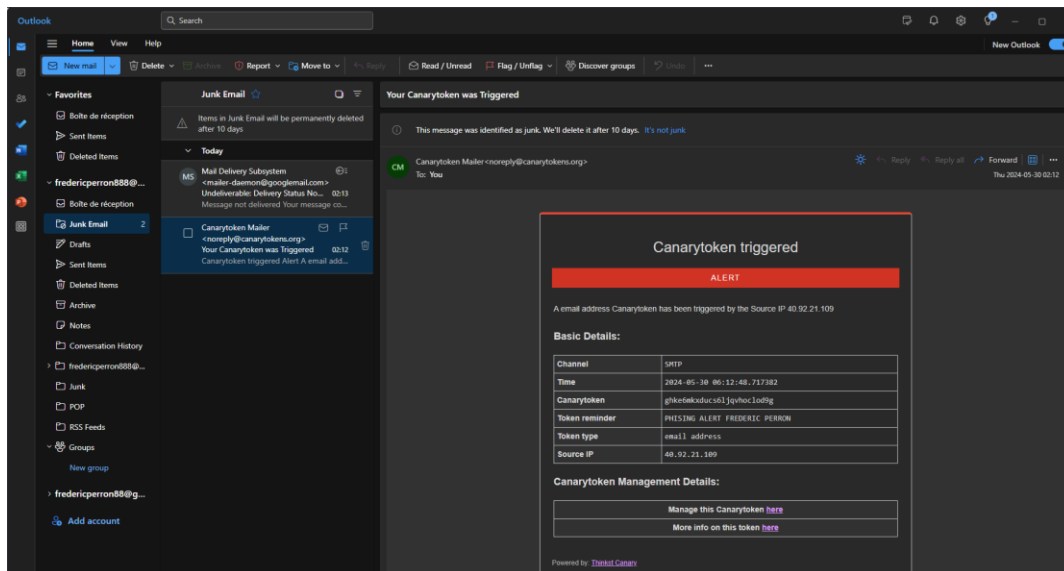
We begin by generating the token on canarytokens.org.



We then send an email from our own address (or any address) to verify whether the alert is triggered.



After a few minutes, we receive a notification confirming that the email was received. *See image below*



While this may seem simple, this token has many practical security applications. Here are two scenarios where it can be highly effective:

Detection of Sensitive Data Leaks:

By embedding a unique email address in documents, databases, or internal systems, an organization can monitor for potential data exfiltration. Any unauthorized communication sent to this address will trigger an alert, helping detect malicious insiders or suspicious activity.

Monitoring Interactions with External Partners:

Organizations can assign unique email addresses when communicating with external partners such as vendors or clients. This enables active monitoring of those interactions and helps identify any abnormal or unauthorized communication patterns.


```
, const void*, long unsigned int)' reading 6 bytes from a region of size 4 [-Wstringop-overflow=]
```

```

../src/sys
ient.o -I.
-I. -I../
39855/tcp filtered unknown
39856/tcp filtered unknown
39857/tcp filtered unknown
39858/tcp filtered unknown
39859/tcp filtered unknown
39860/tcp filtered unknown
47761/tcp open unknown
47762/tcp open unknown
47763/tcp open unknown

Nmap done: 1 IP address (1 host up) scanned in 4.44 seconds
threat@ubuntu:~$ sudo nmap --script s7-info.nse -p 102 192.168.64.160
Starting Nmap 7.80 ( https://nmap.org ) at 2024-06-06 14:23 PDT
Nmap scan report for ubuntu (192.168.64.160)
Host is up (0.000076s latency).

PORT      STATE SERVICE
102/tcp   open  s7-tsap
| s7- info:
|   Module: 6ES7 315-2EH14-0AB0
|   Basic Hardware: 6ES7 315-2EH14-0AB0
|   Version: 3.2.6
|   System Name: SNAP7-SERVER
|   Module Type: CPU 315-2 PN/DP
|   Serial Number: S C-C2UR28922012
|_  Copyright: Original Siemens Equipment
Service Info: Device: specialized

Nmap done: 1 IP address (1 host up) scanned in 0.24 seconds
threat@ubuntu:~$

```

- c. In snort.conf, define a new variable containing the IP addresses of the industrial systems, including the IP of your simulator (e.g., ICS_NET) (2.5p)

```

# Note to Debian users: this value is overridden when starting
# up the Snort daemon through the init.d script by the
# value of DEBIAN_SNORT_HOME_NET s defined in the
# /etc/snort/snort.debian.conf configuration file
#
lpvar HOME_NET any

# Set up the external network addresses. Leave as "any" in most situations
lpvar EXTERNAL_NET any
# If HOME_NET is defined as something other than "any", alternative, you can
# use this definition if you do not want to detect attacks from your internal
# IP addresses:
lpvar ICS_NET 192.168.64.160

# List of DNS servers on your network
lpvar DNS_SERVERS $HOME_NET

# List of SMTP servers on your network
lpvar SMTP_SERVERS $HOME_NET

# List of web servers on your network
lpvar HTTP_SERVERS $HOME_NET

# List of sql servers on your network
lpvar SQL_SERVERS $HOME_NET

```

- d. In snort.conf, define a new variable containing the ports used by industrial systems, including 102 (e.g., ICS_PORTS) (2.5p)

```

# List of ftp servers on your network
lpvar FTP_SERVERS $HOME_NET

# List of sip servers on your network
lpvar SIP_SERVERS $HOME_NET

portvar ICS_PORTS 102

# List of ports you run web servers on
portvar HTTP_PORTS [80,81,311,383,591,593,901,1220,1414,1741,1830,2301,2381,2809,3037,3128,3702,4343,4848,5250,6988,

# List of ports you want to look for SHELLCODE on.
portvar SHELLCODE_PORTS !80

# List of ports you might see oracle attacks on
portvar ORACLE_PORTS 1024:

```

- e. In snort.conf, define a new variable containing the IP addresses of the engineering systems authorized to communicate with the industrial systems (e.g., INGENIERIE_NET) (2.5p)


```
# Set up the external network addresses. Leave as "any" in most situations
# If HOME_NET is defined as something other than "any", alternative, you can
# use this definition if you do not want to detect attacks from your internal
# IP addresses:
#lpvar EXTERNAL_NET $HOME_NET
#lpvar ICS_NET 192.168.64.160
lpvar INGENIERIE_NET 192.168.64.161

# List of DNS servers on your network
lpvar DNS_SERVERS $HOME_NET

# List of SMTP servers on your network
lpvar SMTP_SERVERS $HOME_NET

# List of web servers on your network
lpvar HTTP_SERVERS $HOME_NET

# List of sql servers on your network
lpvar SQL_SERVERS $HOME_NET

# List of telnet servers on your network
```

- f. Create a **single Snort rule** that will detect
- Traffic to an industrial system (IP and port defined via variables) (5p)
 - Traffic originating from a system other than the engineering systems (using variables – ! before a variable means NOT) (5p)
 - The alert message must include your name (5p)

```
threat@ubuntu: /etc/snort/rules
threat@ubuntu: /etc/snort/rules
threat@ubuntu: /etc/snort
CPU nano 4.8
# $Id: local.rules,v 1.11 2004/07/25 20:11:44 bmc Exp $
# LOCAL RULES
# This file intentionally does not come with signatures. Put your local
# additions here.
alert icmp any any -> $HOME_NET any (msg: "ICMP test"; sid:1000001; rev:1; classtype:icmp-event;)
alert tcp !$INGENIERIE_NET any -> $ICS_NET $ICS_PORTS (msg:"Alert: Traffic to ICS from non-engineering system - Frederic Perron"; sid:1000002; rev:1);
```

- g. Generate traffic that will trigger the rule and demonstrate that the rule was hit

```
(kali@kali)-[~]
$ sudo hping3 -S -p 102 192.168.64.160 -a 192.68.64.155

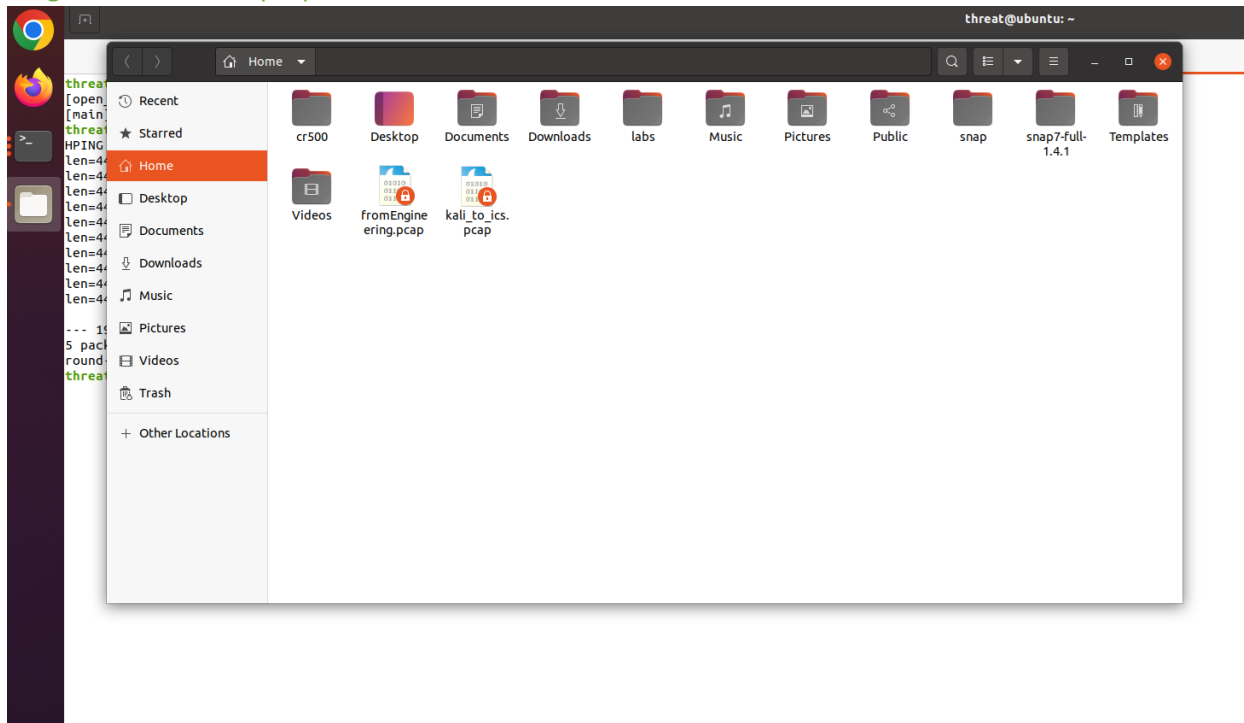
HPING 192.168.64.160 (eth0 192.168.64.160): S set, 40 headers + 0 data bytes
```

Here is the proof below showing that the rule was triggered:

```
Preprocessor Object: SF_GTP Version 1.1 <Build 1>
Commencing packet processing (pid=54343)
06/06-23:11:27.419312 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] [TCP] 192.68.64.155:1624 -> 192.168.64.160:102
06/06-23:11:27.419579 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] [TCP] 192.68.64.155:1624 -> 192.168.64.160:102
06/06-23:11:28.420132 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] [TCP] 192.68.64.155:1625 -> 192.168.64.160:102
06/06-23:11:28.420372 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] [TCP] 192.68.64.155:1625 -> 192.168.64.160:102
06/06-23:11:29.421289 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] [TCP] 192.68.64.155:1626 -> 192.168.64.160:102
06/06-23:11:29.421490 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] [TCP] 192.68.64.155:1626 -> 192.168.64.160:102
06/06-23:11:30.422170 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] [TCP] 192.68.64.155:1627 -> 192.168.64.160:102
06/06-23:11:30.422374 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] [TCP] 192.68.64.155:1627 -> 192.168.64.160:102
06/06-23:11:31.423131 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] [TCP] 192.68.64.155:1628 -> 192.168.64.160:102
06/06-23:11:31.423354 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] [TCP] 192.68.64.155:1628 -> 192.168.64.160:102
06/06-23:11:32.424825 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] [TCP] 192.68.64.155:1629 -> 192.168.64.160:102
06/06-23:11:32.425020 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] [TCP] 192.68.64.155:1629 -> 192.168.64.160:102
06/06-23:11:33.425245 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] [TCP] 192.68.64.155:1630 -> 192.168.64.160:102
06/06-23:11:33.425449 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] [TCP] 192.68.64.155:1630 -> 192.168.64.160:102
06/06-23:11:34.425699 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] [TCP] 192.68.64.155:1631 -> 192.168.64.160:102
06/06-23:11:34.425922 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] [TCP] 192.68.64.155:1631 -> 192.168.64.160:102
06/06-23:11:35.109286 ** [1:1917:6] SCAN UPnP service discover attempt ** [Classification: Detection of a Network Scan] [Priority: 3] [UDP] 192.168.64.1:62261 -> 239.255.255.250:1900
06/06-23:11:35.426147 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] [TCP] 192.68.64.155:1632 -> 192.168.64.160:102
06/06-23:11:35.426338 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] [TCP] 192.68.64.155:1632 -> 192.168.64.160:102
06/06-23:11:36.114194 ** [1:1917:6] SCAN UPnP service discover attempt ** [Classification: Detection of a Network Scan] [Priority: 3] [UDP] 192.168.64.1:62261 -> 239.255.255.250:1900
```

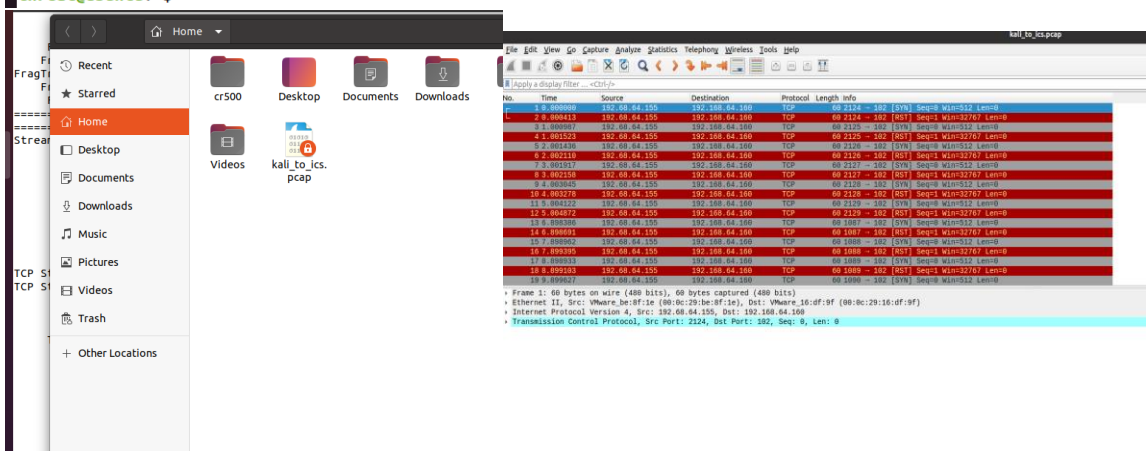
- Use tcpdump with a filter to capture only traffic between a source IP from the engineering network → destination IP in ICS → destination ICS port (2.5p)

I performed part **b** first. I used tcpdump as shown in part b, but with an IP within the **INGENIERIE_NET** range to obtain both pcap files.



- b. Use tcpdump with a filter to capture only traffic between a source IP outside the engineering network → destination IP in ICS → destination ICS port (2.5p)

```
threat@ubuntu:~$ sudo tcpdump -i ens33 src 192.68.64.155 and dst 192.168.64.160 and dst port 102 -w kali_to_ics.pcap
tcpdump: listening on ens33, link-type EN10MB (Ethernet), capture size 262144 bytes
^C34 packets captured
36 packets received by filter
0 packets dropped by kernel
threat@ubuntu:~$
```



- c. Using the pcaps from the previous two steps, test your rule (5p)

```

threat@ubuntu:~$ sudo snort -r /etc/snort.pcap -c /etc/snort/snort.conf -A console
[snort] password for threat:
Running in IDS mode

--= Initializing Snort ==--
Initializing Output Plugins!
Initializing Preprocessor!
Initializing Plug-Ins!
Parsing Rules File /etc/snort/snort.conf
Portvar 'ICL_PORTS' defined : [ 102 ]
Portvar 'HTTP_PORTS' defined : [ 80,81,110,143,311,383,591,593,901,1220,1434,1741,1830,2301,2381,2809,3037,3128,3702,4343,4848,5250,6988,7000,7001,7144,7145 ]
Portvar 'SHELLCODE_PORTS' defined : [ 0:79,81:5535 ]
Portvar 'ORACLE_PORTS' defined : [ 1024:6535 ]
Portvar 'SSH_PORTS' defined : [ 22 ]
Portvar 'FTP_PORTS' defined : [ 21,2300,3535 ]
Portvar 'SIP_PORTS' defined : [ 5060:5061,5069 ]
Portvar 'FILE_DATA_PORTS' defined : [ 0:80,81,110,143,311,383,591,593,901,1220,1434,1741,1830,2301,2381,2809,3037,3128,3702,4343,4848,5250,6988,7000:70 ]
Portvar 'ICL_PORTS' defined : [ 102 ]
Portvar 'ICL_PORTS' defined : [ 1024:6535 ]
Portvar 'FTP_PORTS' defined : [ 21,23,252,3386 ]
Detection:
Search-Method = AC-Full-Q
Split Any/Any group = enabled
Search-Method-Optimizations = enabled
Minimum pattern length = 26
Tagged Packet Limit: 256
Loading dynamic engine /usr/lib/snort_dynamicengine/libsf_engine.so... done
Loading all dynamic detection libs from /usr/lib/snort_dynamicrules...
Warning: No dynamic libraries found in directory /usr/lib/snort_dynamicrules.
Finished loading all dynamic detection libs from /usr/lib/snort_dynamicrules
Loading all dynamic preprocessor libraries from /usr/lib/snort_dynamicpreprocessor...
Loading dynamic preprocessor library /usr/lib/snort_dynamicpreprocessor/libsf_gtp_preproc.so... done
Loading dynamic preprocessor library /usr/lib/snort_dynamicpreprocessor/libsf_dns_preproc.so... done
Loading dynamic preprocessor library /usr/lib/snort_dynamicpreprocessor/libsf_smtp_preproc.so... done
Loading dynamic preprocessor library /usr/lib/snort_dynamicpreprocessor/libsf_dce2_preproc.so... done
Loading dynamic preprocessor library /usr/lib/snort_dynamicpreprocessor/libsf_slp_preproc.so... done
Loading dynamic preprocessor library /usr/lib/snort_dynamicpreprocessor/libsf_ftptelnet_preproc.so... done
Loading dynamic preprocessor library /usr/lib/snort_dynamicpreprocessor/libsf_dmp3_preproc.so... done
Loading dynamic preprocessor library /usr/lib/snort_dynamicpreprocessor/libsf_pop_preproc.so... done
Loading dynamic preprocessor library /usr/lib/snort_dynamicpreprocessor/libsf_lmtp_preproc.so... done
Loading dynamic preprocessor library /usr/lib/snort_dynamicpreprocessor/libsf_snmp_preproc.so... done
Loading dynamic preprocessor library /usr/lib/snort_dynamicpreprocessor/libsf_reputation_preproc.so... done
Loading dynamic preprocessor library /usr/lib/snort_dynamicpreprocessor/libsf_nsl_preproc.so... done
Loading dynamic preprocessor library /usr/lib/snort_dynamicpreprocessor/libsf_modbus_preproc.so... done

```

```

threat@ubuntu:~$
Copyright (c) 2014 Cisco and/or its affiliates. All rights reserved.
Copyright (c) 1998-2013 Sourcefire, Inc., et al.
Using libpcap version 1.9.1 (with PACKET_V3)
Using PDBE version: 8.39 2016-06-14
Using ZLIB version: 1.2.11

Rules Engine: SF_SNORT_DETECTION_ENGINE Version 2.4 <Build 1>
Preprocessor Object: SF_MODBUS Version 1.1 <Build 1>
Preprocessor Object: SF_SALPP Version 1.1 <Build 1>
Preprocessor Object: SF_REPUTATION Version 1.1 <Build 1>
Preprocessor Object: SF_SDF Version 1.1 <Build 1>
Preprocessor Object: SF_SSH Version 1.1 <Build 3>
Preprocessor Object: SF_INAP Version 1.0 <Build 1>
Preprocessor Object: SF_DMP3 Version 1.1 <Build 1>
Preprocessor Object: SF_FTPTELNET Version 1.2 <Build 13>
Preprocessor Object: SF_POP Version 1.0 <Build 1>
Preprocessor Object: SF_SIP Version 1.1 <Build 1>
Preprocessor Object: SF_DCEPC2 Version 1.0 <Build 3>
Preprocessor Object: SF_SNMP Version 1.1 <Build 9>
Preprocessor Object: SF_DNS Version 1.1 <Build 4>
Preprocessor Object: SF_GTP Version 1.1 <Build 1>

Commencing packet processing (pid:57582)
06/06-23:19:47.788900 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:2124 -> 192.168.
06/06-23:19:47.789313 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:2124 -> 192.168.
06/06-23:19:48.789887 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:2125 -> 192.168.
06/06-23:19:48.790423 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:2125 -> 192.168.
06/06-23:19:49.790336 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:2126 -> 192.168.
06/06-23:19:49.791010 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:2126 -> 192.168.
06/06-23:19:50.790817 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:2127 -> 192.168.
06/06-23:19:50.791058 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:2127 -> 192.168.
06/06-23:19:51.791945 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:2128 -> 192.168.
06/06-23:19:51.792178 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:2128 -> 192.168.
06/06-23:19:52.793022 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:2129 -> 192.168.
06/06-23:19:52.793772 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:2129 -> 192.168.
06/06-23:19:54.087286 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:1087 -> 192.168.
06/06-23:19:54.087591 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:1087 -> 192.168.
06/06-23:19:55.087862 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:1088 -> 192.168.
06/06-23:19:55.088295 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:1088 -> 192.168.
06/06-23:19:56.087833 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:1089 -> 192.168.
06/06-23:19:56.088003 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:1089 -> 192.168.
06/06-23:19:57.088527 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:1090 -> 192.168.
06/06-23:19:57.088719 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:1090 -> 192.168.
06/06-23:19:58.089483 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:1091 -> 192.168.
06/06-23:19:58.089766 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:1091 -> 192.168.
06/06-23:19:59.089762 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:1092 -> 192.168.
06/06-23:19:59.089960 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:1092 -> 192.168.
06/06-23:20:00.090726 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:1093 -> 192.168.
06/06-23:20:00.090927 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:1093 -> 192.168.
06/06-23:20:01.091049 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:1094 -> 192.168.
06/06-23:20:01.091211 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:1094 -> 192.168.
06/06-23:20:02.091826 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:1095 -> 192.168.
06/06-23:20:02.091978 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:1095 -> 192.168.
06/06-23:20:03.093456 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:1096 -> 192.168.
06/06-23:20:03.093714 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:1096 -> 192.168.
06/06-23:20:04.094661 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:1097 -> 192.168.
06/06-23:20:04.095020 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.68.64.155:1097 -> 192.168.

=====
Run time for packet processing was 1.1794 seconds
Snort processed 34 packets.
Snort ran for 0 days 0 hours 0 minutes 1 seconds
Pkts/sec: 34
=====
Memory usage summary:
Total non-mapped bytes (arena): 43860160
Bytes in named reontions (hblkhld): 11574144

```

And now the file fromEngineering.pcap :

```
SMTP Preprocessor Statistics
Total sessions : 0
Max concurrent sessions : 0
=====
dcerpc2 Preprocessor Statistics
Total sessions: 0
=====
SIP Preprocessor Statistics
Total sessions: 0
=====
Snort exiting
threat@ubuntu:~$ sudo snort -r fromEngineering.pcap -c /etc/snort/snort.conf -A console
Running in IDS mode

--== Initializing Snort ==--
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file "/etc/snort/snort.conf"
PortVar 'ICS_PORTS' defined : [ 102 ]
PortVar 'HTTP_PORTS' defined : [ 80:81 311 383 591 593 901 1220 1414 1741 1830 2301 2381 2809 3037 3128
88 8899 9000 9060 9080 9090:9091 9443 9999 11371 34443:34444 41080 50002 55555 ]
PortVar 'SHELLCODE_PORTS' defined : [ 0:79 81:65535 ]
PortVar 'ORACLE_PORTS' defined : [ 1024:65535 ]
PortVar 'SSH_PORTS' defined : [ 22 ]
PortVar 'FTP_PORTS' defined : [ 21 2100 3535 ]
PortVar 'SIP_PORTS' defined : [ 5060:5061 5600 ]
PortVar 'FILE_DATA_PORTS' defined : [ 80:81 110 143 311 383 591 593 901 1220 1414 1741 1830 2301 2381 28
8300 8800 8888 8899 9000 9060 9080 9090:9091 9443 9999 11371 34443:34444 41080 50002 55555 ]
PortVar 'GTP_PORTS' defined : [ 2123 2152 3386 ]
Detection:
Search-Method = AC-Full-Q
Split Any/Any group = enabled
Search-Method-Optimizations = enabled
Maximum pattern length = 20
Tagged Packet Limit: 256
Loading dynamic engine /usr/lib/snort_dynamicengine/libsf_engine.so... done
Loading all dynamic detection libs from /usr/lib/snort_dynamicrules...
```

And the rule was also triggered:

```
Copyright (C) 2014 Cisco and/or its affiliates. All rights reserved.
Copyright (C) 1998-2013 Sourcefire, Inc., et al.
Using libpcap version 1.9.1 (with TPACKET_V3)
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.2.11

Rules Engine: SF_SNORT_DETECTION_ENGINE Version 2.4 <Build 1>
Preprocessor Object: SF_MODBUS Version 1.1 <Build 1>
Preprocessor Object: SF_SSHPP Version 1.1 <Build 4>
Preprocessor Object: SF_REPUTATION Version 1.1 <Build 1>
Preprocessor Object: SF_SDF Version 1.1 <Build 1>
Preprocessor Object: SF_SSH Version 1.1 <Build 3>
Preprocessor Object: SF_INAP Version 1.0 <Build 1>
Preprocessor Object: SF_POP Version 1.0 <Build 1>
Preprocessor Object: SF_DWP3 Version 1.1 <Build 1>
Preprocessor Object: SF_FTPTELNET Version 1.2 <Build 13>
Preprocessor Object: SF_SIP Version 1.1 <Build 1>
Preprocessor Object: SF_DICERPC2 Version 1.0 <Build 3>
Preprocessor Object: SF_SMTP Version 1.1 <Build 9>
Preprocessor Object: SF_DNS Version 1.1 <Build 4>
Preprocessor Object: SF_GTP Version 1.1 <Build 1>

Commencing packet processing (pid=58901)
06/06-23:19:47.789900 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:21224 -> 192.168.04.160:102
06/06-23:19:47.789913 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:21224 -> 192.168.04.160:102
06/06-23:19:48.790887 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:21225 -> 192.168.04.160:102
06/06-23:19:48.790423 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:21225 -> 192.168.04.160:102
06/06-23:19:49.790336 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:21226 -> 192.168.04.160:102
06/06-23:19:49.791010 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:21226 -> 192.168.04.160:102
06/06-23:19:50.790817 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:21227 -> 192.168.04.160:102
06/06-23:19:50.791058 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:21227 -> 192.168.04.160:102
06/06-23:19:51.792178 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:21228 -> 192.168.04.160:102
06/06-23:19:52.793002 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:21229 -> 192.168.04.160:102
06/06-23:19:52.793772 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:21229 -> 192.168.04.160:102
06/06-23:19:54.687280 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:1087 -> 192.168.04.160:102
06/06-23:19:54.687591 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:1087 -> 192.168.04.160:102
06/06-23:19:55.687862 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:1088 -> 192.168.04.160:102
06/06-23:19:55.688295 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:1088 -> 192.168.04.160:102
06/06-23:19:56.687833 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:1089 -> 192.168.04.160:102
06/06-23:19:56.688003 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:1089 -> 192.168.04.160:102
06/06-23:19:57.688527 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:1090 -> 192.168.04.160:102
06/06-23:19:57.688719 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:1090 -> 192.168.04.160:102
06/06-23:19:58.689483 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:1091 -> 192.168.04.160:102
06/06-23:19:58.689796 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:1091 -> 192.168.04.160:102
06/06-23:19:59.689762 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:1092 -> 192.168.04.160:102
06/06-23:19:59.689966 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:1092 -> 192.168.04.160:102
06/06-23:20:00.690726 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:1093 -> 192.168.04.160:102
06/06-23:20:00.690927 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:1093 -> 192.168.04.160:102
06/06-23:20:01.691049 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:1094 -> 192.168.04.160:102
06/06-23:20:01.691211 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:1094 -> 192.168.04.160:102
06/06-23:20:02.691026 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:1095 -> 192.168.04.160:102
06/06-23:20:02.691978 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:1095 -> 192.168.04.160:102
06/06-23:20:03.693456 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:1096 -> 192.168.04.160:102
06/06-23:20:03.693714 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:1096 -> 192.168.04.160:102
06/06-23:20:04.694601 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:1097 -> 192.168.04.160:102
06/06-23:20:04.695020 ** [1:1000002:1] Alert: Traffic to ICS from non-engineering system - Frederic Perron ** [Priority: 0] (TCP) 192.08.04.161:1097 -> 192.168.04.160:102
```

Done for this exercise (02).

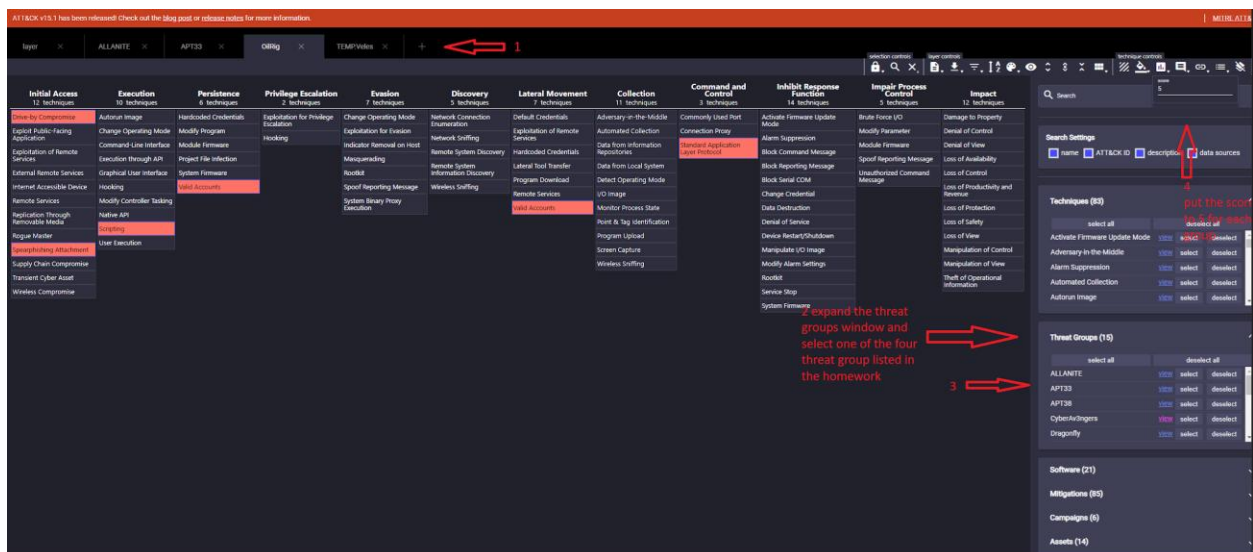
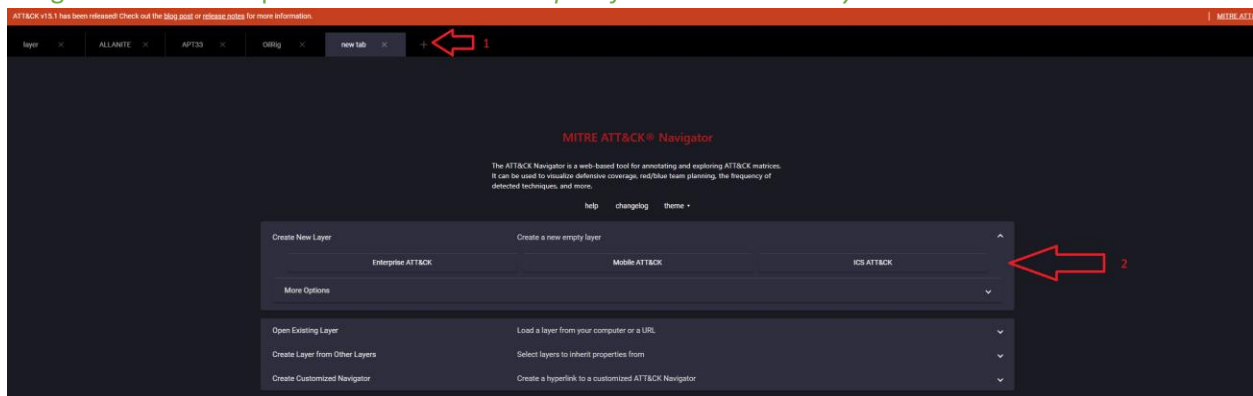
Exercise No 3 (10p) – Mitre Attack Navigator

You work for Hydro-Québec and want to identify threat actors active in your domain. You evaluate each actor with a specific score.

Navigate to the <https://mitre-attack.github.io/attack-navigator/> (ICS version) and complete the following tasks:

- Combine the following groups, assigning each a score of 5 points (5p)
 - ALLANITE – 5 points
 - APT33 – 5 points
 - OilRig- 5 points
 - TEMP.Veles– 5 points
- Take a screenshot of your results and include it in the assignment. Export the JSON file associated with the new layer and attach it to the submission (2.5p)
- Find the 4 techniques with the highest combined score (2.5p)

We begin by creating the four layers in the navigator so they can be combined. Each layer is created and assigned a score of 5 points. *Below is an example of how to create a layer*



[illegible]

12 | Page

Technique 1

T1189 - Drive-by Compromise

Description: Compromise of a system through exploitation of vulnerabilities in a web browser or plugin when visiting a malicious or compromised website. This type of attack requires no user interaction beyond visiting the site.

Technique 2

T1566.001 - Spearphishing Attachment

Description: Sending targeted emails with malicious attachments designed to infect the user's system when opened. Attachments may contain macros, exploits, or malicious scripts.

Technique 3

T1064 - Scripting

Description: Use of scripts and command interpreters to execute malicious commands. Scripts can automate tasks, deliver payloads, or bypass existing security mechanisms.

Technique 4

T1078 - Valid Accounts

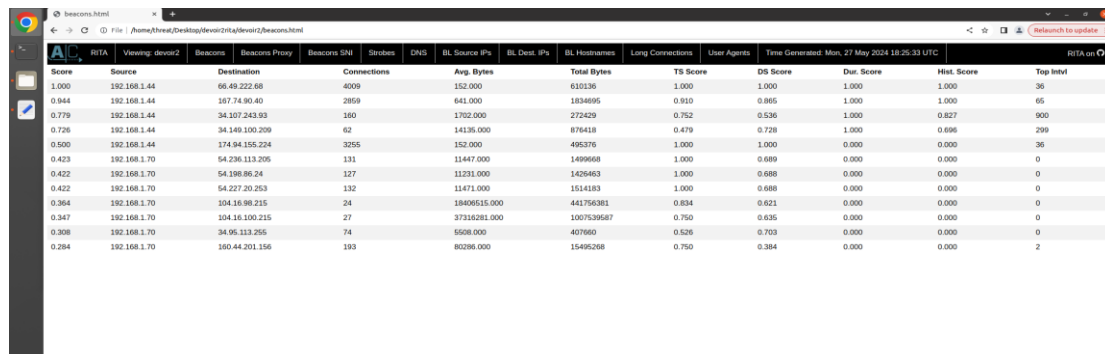
Description: Use of legitimate user accounts to access systems and networks. Attackers may obtain these accounts through phishing, credential theft, or by exploiting weaknesses in account management.

Since these techniques share the same highest score, they should all be considered to maintain a comprehensive view of the most relevant threats for Hydro-Québec. Each represents commonly used tactics by the analyzed threat groups, highlighting priority attack vectors that should be closely monitored and secured.

Exercise No 4 – Zeek & RITA (5p)

Using the instructions from Lab 4 – Course 6, the Hunting machine, and the files in the *devoir2rita* folder, perform an analysis using RITA.

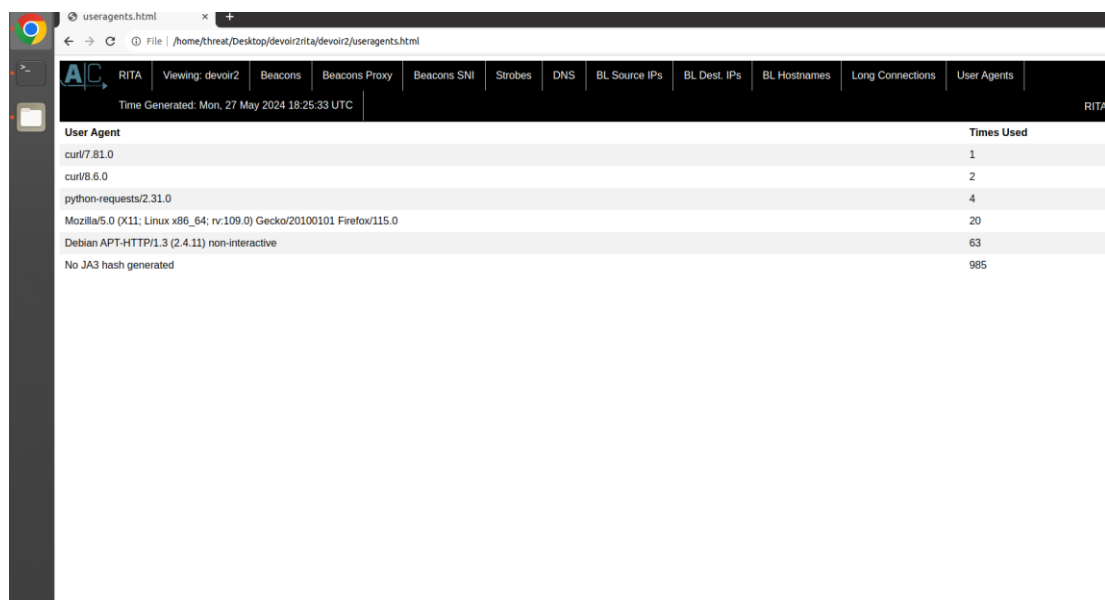
We begin by extracting the HTML files from *devoir2rita* to the Desktop so they can be accessed via a browser. In the **beacons** section, the following results and analysis are observed:



Score	Source	Destination	Connections	Avg. Bytes	Total Bytes	TS Score	DS Score	Dur. Score	Hist. Score	Top Intvl
1.000	192.168.1.44	66.49.222.68	4009	152.000	610136	1.000	1.000	1.000	1.000	36
0.944	192.168.1.44	167.74.90.40	2859	641.000	1834695	0.910	0.865	1.000	1.000	65
0.779	192.168.1.44	34.107.243.93	160	1702.000	272429	0.752	0.536	1.000	0.827	900
0.726	192.168.1.44	34.149.100.209	62	14135.000	876418	0.479	0.728	1.000	0.696	299
0.500	192.168.1.44	174.94.185.224	3255	152.000	496376	1.000	1.000	0.000	0.000	36
0.423	192.168.1.70	54.236.113.205	131	11447.000	1499668	1.000	0.689	0.000	0.000	0
0.422	192.168.1.70	54.198.86.24	127	11231.000	1426463	1.000	0.688	0.000	0.000	0
0.422	192.168.1.70	54.227.20.253	132	11471.000	1514183	1.000	0.688	0.000	0.000	0
0.364	192.168.1.70	104.16.98.215	24	1840615.000	441756381	0.834	0.621	0.000	0.000	0
0.347	192.168.1.70	104.16.100.215	27	37316281.000	1007539587	0.750	0.635	0.000	0.000	0
0.308	192.168.1.70	34.95.113.255	74	5508.000	407860	0.526	0.703	0.000	0.000	0
0.284	192.168.1.70	160.44.201.156	193	80296.000	15495268	0.750	0.384	0.000	0.000	2

We can infer that **192.168.1.44** is highly suspicious. This pattern is consistent with a backdoor beaconing behavior, where an infected host periodically reconnects to receive commands from an attacker at regular intervals. The high number of connections is abnormal and warrants further investigation.

We can also observe many User Agents labeled “**No JA3 hash generated**”, appearing **985 times**, compared to only **63** for the next most frequent entry. This discrepancy is also highly suspicious and may indicate non-standard or obfuscated traffic.

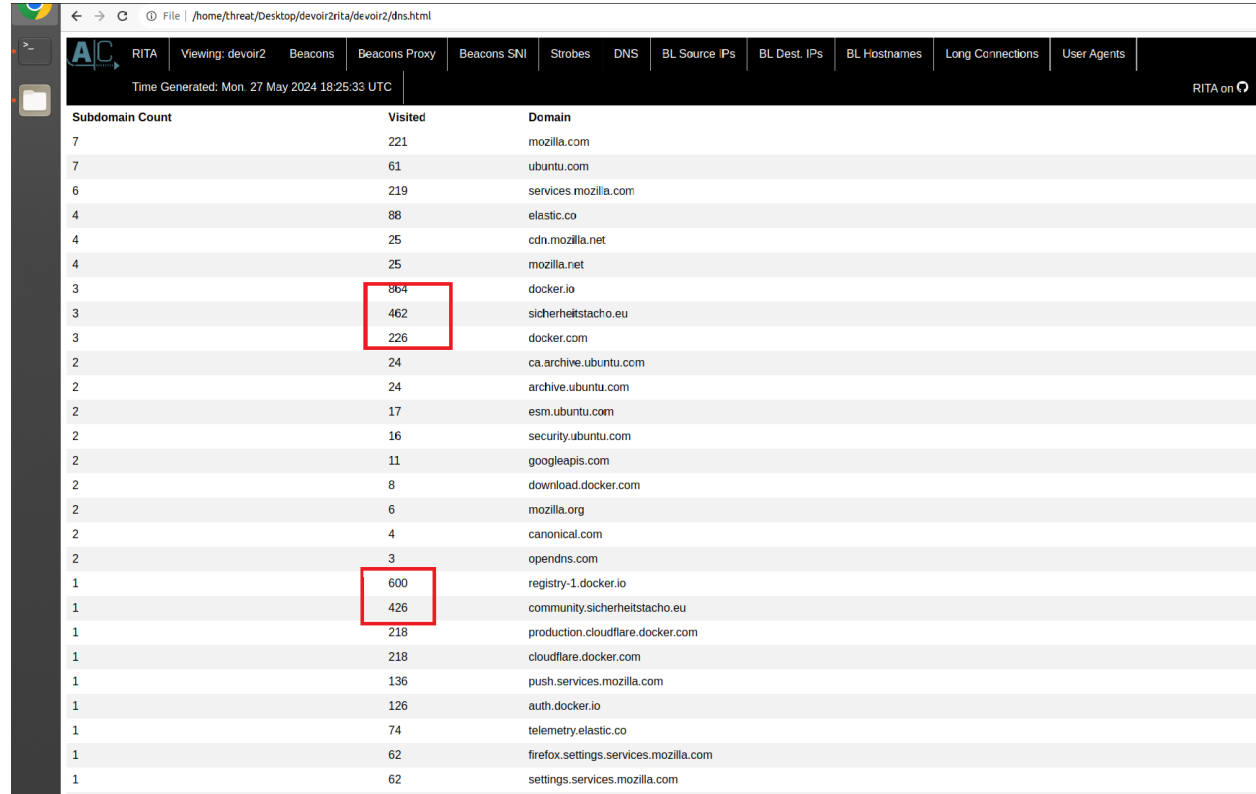


User Agent	Times Used
curl/7.81.0	1
curl/8.6.0	2
python-requests/2.31.0	4
Mozilla/5.0 (X11; Linux x86_64; rv:109.0) Gecko/20100101 Firefox/115.0	20
Debian APT-HTTP/1.3 (2.4.11) non-interactive	63
No JA3 hash generated	985

Next, we further investigate logs using **Zeek**, identifying the most frequently contacted domains during the same timeframe. Additionally, some connections show long durations (with a duration score of

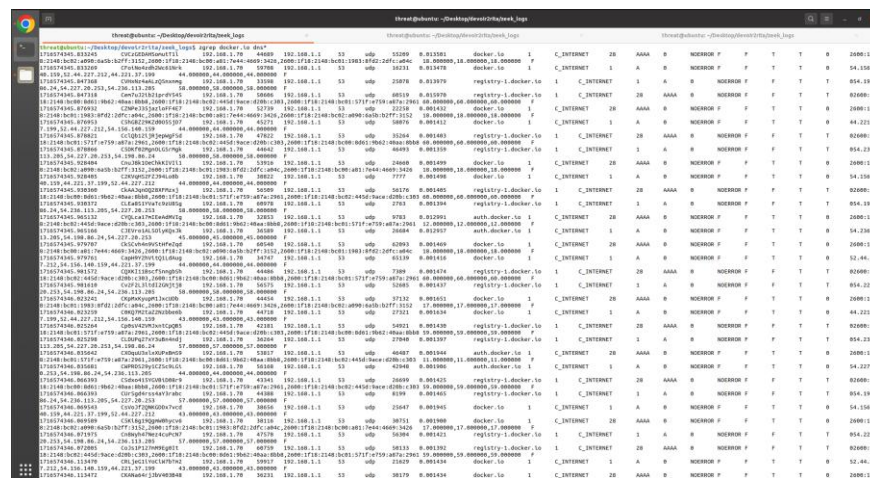
1.000 for certain IPs), indicating persistent connections. This behavior is typical of command-and-control (C2) channels that remain open for continuous command execution or data exfiltration.

With **Zeek** and **zgrep**, we can further investigate each DNS packet per domain. If a domain appears suspicious—such as **sicherstacho.eu** or **docker.io**—we can filter it directly in the terminal using **zgrep** to streamline the analysis.



Subdomain Count	Visited	Domain
7	221	mozilla.com
7	61	ubuntu.com
6	219	services.mozilla.com
4	88	elastic.co
4	25	cdn.mozilla.net
4	25	mozilla.net
3	864	docker.io
3	462	sicherstacho.eu
3	226	docker.com
2	24	ca.archive.ubuntu.com
2	24	archive.ubuntu.com
2	17	esm.ubuntu.com
2	16	security.ubuntu.com
2	11	googleapis.com
2	8	download.docker.com
2	6	mozilla.org
2	4	canonical.com
2	3	opendns.com
1	600	registry-1.docker.io
1	426	community.sicherstacho.eu
1	218	production.cloudflare.docker.com
1	218	cloudflare.docker.com
1	136	push.services.mozilla.com
1	126	auth.docker.io
1	74	telemetry.elastic.co
1	62	firefox.settings.services.mozilla.com
1	62	settings.services.mozilla.com

During this investigation, I identified an unusual domain containing a “**listbot**” pattern in the DNS queries, which stands out as potentially suspicious. *Below is a screenshot of the three domains that were analyzed*



IP Address	Domain	Query Type
192.168.1.1	mozilla.com	A
192.168.1.1	ubuntu.com	A
192.168.1.1	services.mozilla.com	A
192.168.1.1	elastic.co	A
192.168.1.1	cdn.mozilla.net	A
192.168.1.1	mozilla.net	A
192.168.1.1	docker.io	A
192.168.1.1	sicherstacho.eu	A
192.168.1.1	docker.com	A
192.168.1.1	ca.archive.ubuntu.com	A
192.168.1.1	archive.ubuntu.com	A
192.168.1.1	esm.ubuntu.com	A
192.168.1.1	security.ubuntu.com	A
192.168.1.1	googleapis.com	A
192.168.1.1	download.docker.com	A
192.168.1.1	mozilla.org	A
192.168.1.1	canonical.com	A
192.168.1.1	opendns.com	A
192.168.1.1	registry-1.docker.io	A
192.168.1.1	community.sicherstacho.eu	A
192.168.1.1	production.cloudflare.docker.com	A
192.168.1.1	cloudflare.docker.com	A
192.168.1.1	push.services.mozilla.com	A
192.168.1.1	auth.docker.io	A
192.168.1.1	telemetry.elastic.co	A
192.168.1.1	firefox.settings.services.mozilla.com	A
192.168.1.1	settings.services.mozilla.com	A

Exercise No 5 – SiLK (20p)

Objective: Inspect the flow data in *ex5abcd.silk* to analyze network behavior.

Description : Lire les enregistrements de flux pour examiner les différents aspects du comportement du réseau du trafic capturé

- a) Description : Combien d'enregistrements de flux sont dans le fichier **ex5abcd.silk**?

Hint : Use the **rwfilter** command to read the *ex5abcd.silk* file and include all possible protocol numbers using the partitioning switch **--proto=0-255**, and specify **--print-stat** to print statistics. The **--print-stat** option is an output switch. (3p)

Answer : **75**

```
threat@ubuntu:~/Desktop/devoir2SiLK$ rwfilter ex5abcd.silk --proto=0-255 --print-stat
Files      1.  Read      75.  Pass      75.  Fail      0.
threat@ubuntu:~/Desktop/devoir2SiLK$
```

- b) **Description:** Find the two flow records associated with the IP address **209.85.227.106** and port **1088**, and answer the following questions after the hints and space to record the SiLK command used.

Hint : Use the **rwfilter** command and specify the partitioning parameters **--any-address** and **--aport**. These parameters search for traffic in both directions within the flows. Do not forget to provide the input file *ex5abcd.silk* for the **rwfilter** command, and use the output parameter **--pass=stdout** to pass the extracted flows to the next command for conversion to ASCII output.

Hint : Pipe the output of **rwfilter** to the **rwcut** command to convert the binary output into ASCII and display all flow fields.

```
threat@ubuntu:~/Desktop/devoir2SiLK$ rwfilter ex5abcd.silk --any-address=209.85.227.106 --aport=1088 --pass=stdout | rwcut --fields=sip,dip,sport,dport,stime,proto,packets,bytes
sip|dip|sport|dport|stime|proto|packets|bytes|
10.0.3.15|209.85.227.106|80|80|2010/01/01T00:01:26.109|6|5|731|
209.85.227.106|10.0.3.15|80|80|2010/01/01T00:01:26.169|6|3|553|
threat@ubuntu:~/Desktop/devoir2SiLK$
```

Record the SiLK commands used:

- By examining the flow records, which IP do you think is the client? Why? (Look at the ports and start times.)

10.0.3.15 is the client. The flow start time shows that the connection was initiated by 10.0.3.15 (00:01:26.109), followed by a response from 209.85.227.106 (00:01:26.169), which indicates that 10.0.3.15 initiated the connection.

- Which is the server? Why? (Regardez les ports et les heures de début.)

Answer: 209.85.227.106. The source port used is **80**, which is the standard port for HTTP, indicating it is a web server responding to an HTTP request. Port **1088** is an ephemeral port used by the client to receive the response.

- What protocol are they both using?

Answer : TCP

- Which side closed the connection? How do you know? (Look at flags for FIN or RST.)

Answer : The flags indicate how packets are used in the TCP connection. The side that closed the connection is **10.0.3.15**, because the '**R**' (**RST**) flag in its packets indicates that it sent a reset packet, immediately terminating the connection.

- How many packets and bytes did 10.0.3.15 send?

Answer : 5 packets, 731 bytes

- How long did the flow last when 209.85.227.106 is the source IP? (Check duration column)

Answer : 11.163

```
threat@ubuntu:~/Desktop/devotr2SILK$ rfilter ex5abcd.silk --any-address=209.85.227.106 --sport=1088 --pass=stdout | rwcut --fields=sip,dip,sport,dport,stime,proto,packets,bytes,flags,dur
      sip|                dip|sport|dport|stime|proto| packets|    bytes|    flags| duration|
      10.0.3.15|          209.85.227.106| 1088|   80|2010/01/01T00:01:26.109|  6|         5|    731| SRPA |    11.223|
      209.85.227.106|          10.0.3.15|   80| 1088|2010/01/01T00:01:26.169|  6|         3|    553| S PA |    11.163|
threat@ubuntu:~/Desktop/devotr2SILK$
```

- On what date and time did these flows start? (Format: YYYY/MM/DDTHH) (8p)

Answer : 2010/01/01T00:01:26.109

```
threat@ubuntu:~/Desktop/devotr2SILK$ rfilter ex5abcd.silk --any-address=209.85.227.106 --sport=1088 --pass=stdout | rwcut --fields=sip,dip,sport,dport,stime,proto,packets,bytes,flags,dur
      sip|                dip|sport|dport|stime|proto| packets|    bytes|    flags| duration|
      10.0.3.15|          209.85.227.106| 1088|   80|2010/01/01T00:01:26.109|  6|         5|    731| SRPA |    11.223|
      209.85.227.106|          10.0.3.15|   80| 1088|2010/01/01T00:01:26.169|  6|         3|    553| S PA |    11.163|
threat@ubuntu:~/Desktop/devotr2SILK$
```

- c) Description: What are the top five senders (source IPs) by number of bytes?

Hint: Use the **rwstats** command to read the *ex5abcd.silk* file and specify **--fields=sip**, **output --bytes**, and **--count=5**. (3p)

Answer: See image below for the 5 IPs

```
threat@ubuntu:~/Desktop/devoir2SILK$ rwestats ex5abcd.silk --fields=sip --values=bytes --count=5
INPUT: 75 Records for 19 Bins and 281672 Total Bytes
OUTPUT: Top 5 Bins by Bytes
```

sip	Bytes	Nbytes	cumul_%
192.168.56.52	95397	33.868116	33.868116
64.236.114.1	87414	31.033969	64.902085
10.0.4.15	22190	7.877957	72.780042
192.168.56.50	19195	6.814664	79.594706
10.0.3.151	18799	6.674075	86.268781



- d) Description: What are all the unique UDP destination ports??

Hint: Use the **rwfilter** command with the **ex5abcd.silk** file to extract all UDP flows by using **--proto=17** as a partitioning parameter, and pass the flows for further processing.

Hint: Pipe the output of **rwfilter** to the **rwuniq** command to display the destination ports using **-fields=4** (the destination port field). (3p)

See image below for the 6 ports

```
threat@ubuntu:~/Desktop/devoir2SILK$ rwfilter ex5abcd.silk --proto=17 --pass=stdout | rwuniq --fields=dport
```

dPort	Records
1029	2
137	4
53	2
138	4
67	1
68	4



- e) Description: Using the file **ex5e.silk**, run a command to display the top 10 groupings by total bytes. (3p)

See image below

```
threat@ubuntu:~/Desktop/devoir2SILK$ rwestats --bytes --fields=bytes --count=10 ex5e.silk | sort -rn
```

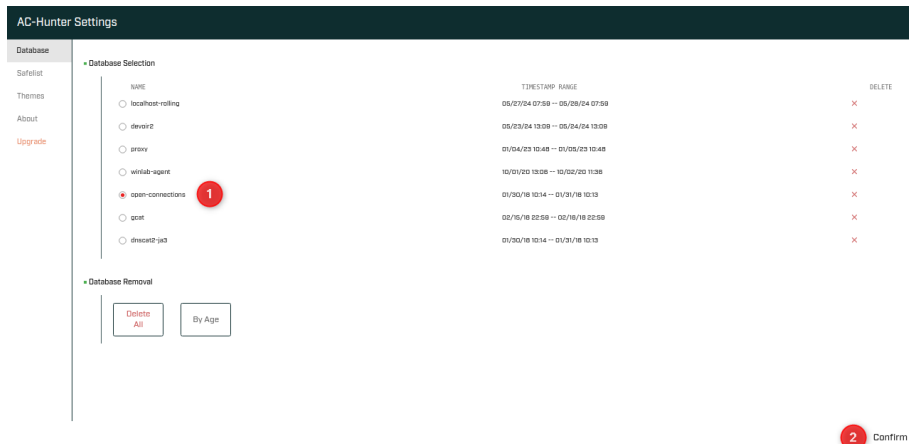
411	411	7.539901	84.149697
264	2376	43.588332	43.588332
229	229	4.201064	88.350761
156	156	2.861860	95.028435
104	208	3.815814	92.166575
84	84	1.541002	96.569437
76	76	1.394240	97.963676
44	440	8.071913	76.609796
40	800	14.676206	58.264539
28	560	10.273344	68.537883

```
OUTPUT: Top 10 Bins by Bytes
INPUT: 68 Records for 12 Bins and 5451 Total Bytes
bytes| Bytes| %Bytes| cumul_%|
threat@ubuntu:~/Desktop/devoir2SILK$
```

Exercise No 6 – Threat Hunting investigation. max 6 pages (10p)

Objectives: Identify compromised machines. Using the Hunting machine, open Chrome and connect to the AC-Hunter web interface at <https://127.0.0.1/home>. Change the dataset and select **open-connections**.

Select open-connections



Perform an analysis of the different indicators.



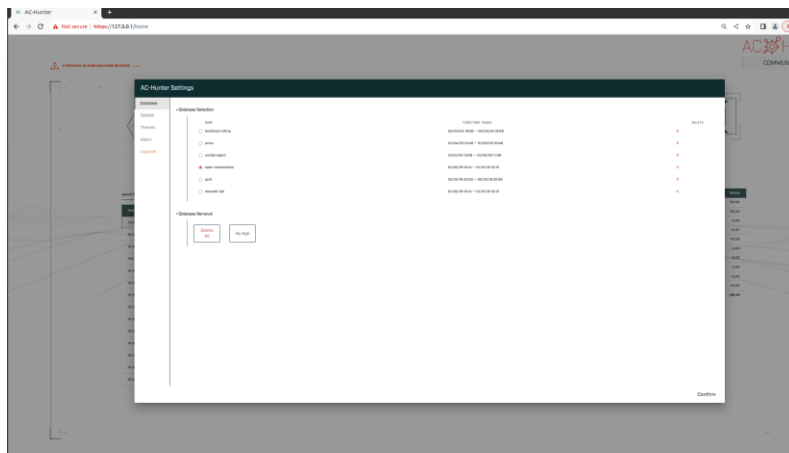
Use various AC-Hunter features such as **filtering out indicators** and **deep dive...**

Follow the investigation approach shown in the following video:

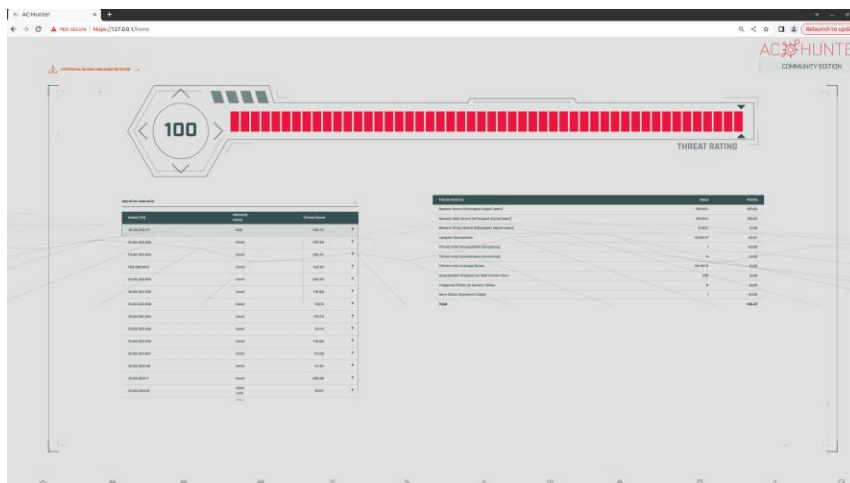
<https://www.youtube.com/watch?v=of6KG3nfOFI>

Grading will be based on the depth of investigation.

We begin by selecting the **open-connections** dataset.



We then click **confirm** in the bottom right. When returning to the main page, we can see the new **Threat Rating of 100**.



We observe several IP addresses with high threat scores, but one stands out: **10.55.100.111**.

This machine has a significantly higher **beacon connection count** compared to others.

Here's what it means and what is a Beacon/C2 attack:

What does 20,000 beacon connections mean?

Having 20,000 beacon connections indicates very frequent and regular communication activity between the IP address 10.55.100.111 and an external server. This reflects abnormal and potentially malicious behavior, typical of communications between a compromised system and a command-and-control (C2) server. This high volume of connections strongly suggests that the IP address is engaged in beaconing activity, where malware or another malicious program maintains persistent communication with a C2 server to receive instructions or transmit data. A high score of 20,000 beacon connections for the IP address 10.55.100.111 is a clear indication of suspicious network activity, suggesting regular communication with a C2 server. This requires in-depth investigation and immediate action to isolate

and secure the compromised system. Understanding how beaconing attacks and C2 infrastructures operate helps improve the protection of systems and networks against these persistent threats.

Based on the number of connections observed, it can be inferred that this IP address is likely the compromised machine used by the attacker to communicate with other systems on the network. In the graph, we can observe 20,000 connections within one second, followed by 200–300 connections per second. What could this indicate?

Continuous communication with the C2 server :

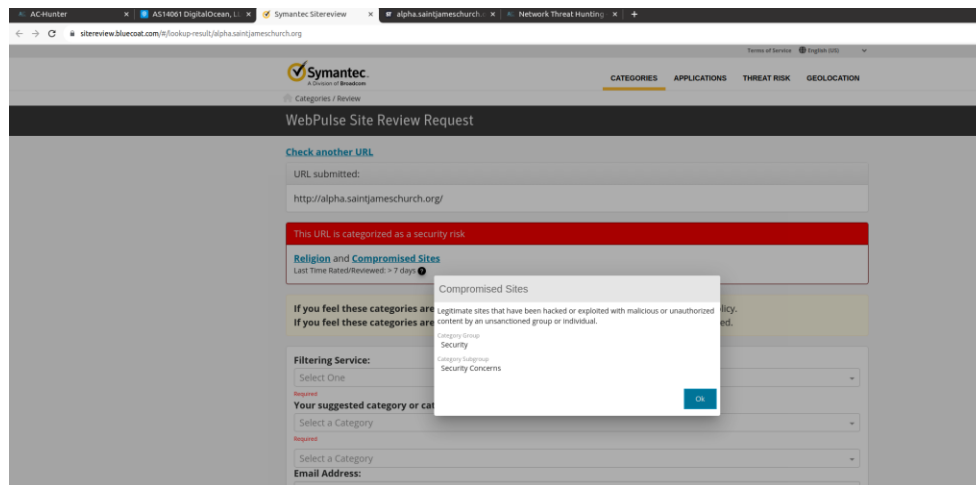
Explanation: After the initial spike, the consistent pattern of 200 to 300 connections per second could represent continuous and stable communication with the C2 server. The malware may be sending and receiving commands, updating its status, or continuously exfiltrating data.

Data Exfiltration or Monitoring:

Explanation: This sustained rate of connections may also indicate a data exfiltration process or continuous monitoring. The malware could be transmitting sensitive data or monitoring network activity and reporting this information back to the C2 server.

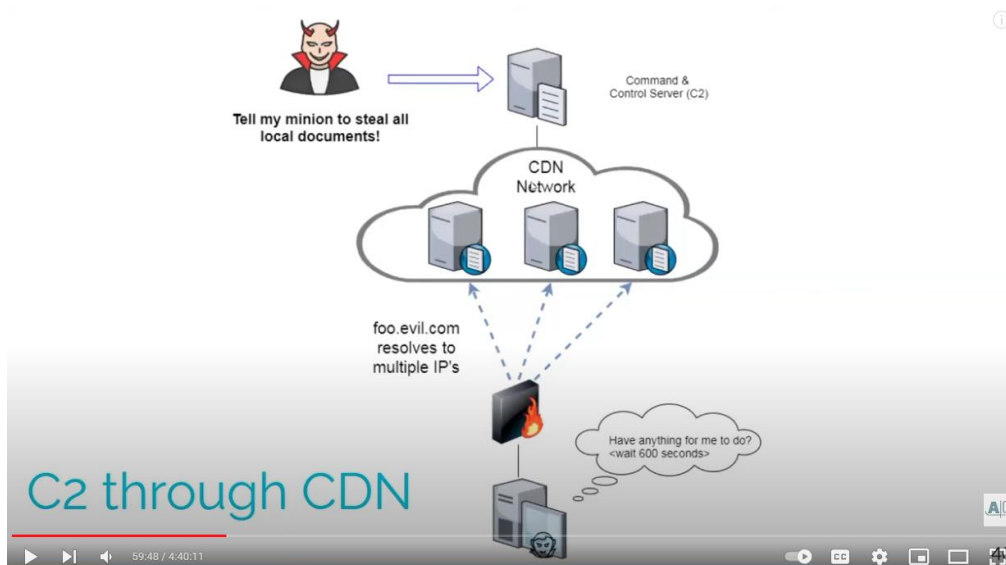
Let us now inspect the Web Beacons section:

When using SiteReview to analyze the FQDN history of the destination associated with these beacon connections from our IP address, we observe several indicators suggesting that the website may be compromised.



What does this mean? It is likely what the attackers are using as their command-and-control (C2) infrastructure.

Explanation: The attackers are using the compromised website as a C2 server. This allows them to communicate with compromised systems while leveraging an apparently legitimate domain to mask their activity.



As mentioned in the video, another suspicious indicator could be the queried FQDN returning no results.

Now, for the “strokes” indicator, here is the analysis:

Src	Src Network Name	Dst	Dst Network Name	Connection Count	Dst FQDN
192.168.88.2	Carol	165.227.88.15	Public	108928	wpchungassitant.com

165.227.88.15

General Information

Hostnames	wpchungassitant.com www.wpchungassitant.com
Domains	CHUNGASSITANT.COM
Cloud Provider	DigitalOcean
Cloud Region	us-nj
Country	United States
City	North Bergen
Organization	DigitalOcean, LLC
ISP	DigitalOcean, LLC
ASN	AS14061

Web Technologies

Mindblown: a blog about philosophy.

Hello world!

Welcome to WordPress. This is your first post.
Edit or delete it, then start writing!

May 7, 2015

Got any book recommendations?

Search

IP Source (Src IP) : 192.168.88.2

Name of Network Source (Src Network Name) : Carol

IP Destination (Dst IP) : 165.227.88.15

Name of Network Destination (Dst Network Name) : Public

Connection Count : 108858

Name of FQDN (Dst FQDN) : badddns.r-1x.com

Analysis :

IP Source (192.168.88.2) :

This is a private IP address (RFC 1918), typically used for internal network addressing. It does not provide much information about the nature of the threat on its own.

Destination IP (165.227.88.15):

This is a public IP address. To obtain more information, a WHOIS lookup or consultation of threat intelligence sources would be required.

Destination Fully Qualified Domain Name (badddns.r-1x.com):

The domain name "badddns.r-1x.com" is highly suspicious. The prefix "bad" suggests potential malicious intent. Additionally, "ddns" refers to dynamic DNS, which is often used by cybercriminals to evade detection by frequently changing the IP addresses associated with the domain.

Number of Connections (108858):

The very high number of connections indicates an abnormally large volume of traffic between the source and the destination. This may suggest that the source machine is infected with malware and is frequently communicating with a command-and-control server hosted at the destination.

Suspicious Indicators:

A high number of connections is abnormal and suggests potentially malicious activity. The domain "badddns.r-1x.com" is a strong indicator of malicious behavior.

For the “long connections” section:

Source (Src) and Destination (Dst):

All source IP addresses belong to the same range (10.55.100.x), indicating they originate from the same internal network named "Carol". The destination IP addresses are all public.

Protocol and Port:

The protocol used is primarily TCP on port 443, which is typical for HTTPS traffic.

Duration and Data Size:

The connections have long durations, ranging from 15 hours to nearly 24 hours, which is unusual for standard HTTPS connections.

The volume of data transferred is relatively low compared to the duration of the connection. Suspicious behavior?

Long and Stable Connections:

HTTPS connections (port 443) with durations close to 24 hours are suspicious. Legitimate HTTPS connections for normal user sessions typically do not last this long.

Data Transfer Volume:

The amount of data transferred is low relative to the connection duration. This may indicate command-and-control (C2) activity, where data exchanges are intermittent but the connection remains open to receive instructions.

Adresse IP de Destination :

The destination IP address 65.52.108.225 is identified as “Microsoft Corp.” This could be legitimate, but it is worth verifying whether this traffic is expected within your environment.

Other destination IP addresses should also be checked to determine whether they belong to legitimate services or are associated with malicious activity.

Recommended Checks

Destination Analysis:

Verify destination IP addresses to determine whether they belong to known and expected services in your environment. As observed using SiteReview, domains such as the Saint Church site show indicators of malicious URLs.

Connection Inspection:

Long-duration HTTPS connections with low data transfer may indicate malware communicating with C2 servers.

Continuous Monitoring:

Set up alerts for long-duration HTTPS connections and for unusually low data volumes relative to connection duration. Perform a deeper analysis of the source hosts to detect potential malware or infections.

Conclusion

Long and stable connections with low data volume over port 443 in AC Hunter may indicate malicious activity, particularly command-and-control (C2) communications. It is essential to verify destination IP addresses and closely monitor these connections.

For the section “threat intel” :

Similar information to the previous indicator is present here. We can observe the same destination address along with its associated bytes transferred and total number of connections. The unusual use of intensive DNS ports and the use of port 53 for TCP connections are suspicious.

Now, using the deep dive tool on the compromised IP address (10.55.100.111), we can see all the addresses and queries made by this host, which facilitates further analysis.

